

The Man-Bites-Dog Phenomenon in Cybersecurity

A Case Study on Hardening Legacy Hardware Communications with the Sony AIBO ERS-7

Presented by:
Alexis Regardin

Overview

01. Introduction

02. Problem statement

03. Threat modeling

04. Secure solutions

05. ROS 2 integration

06. Testing

07. Measurements

08. Learning outcome

09. Future work

01. Introduction

Legacy system

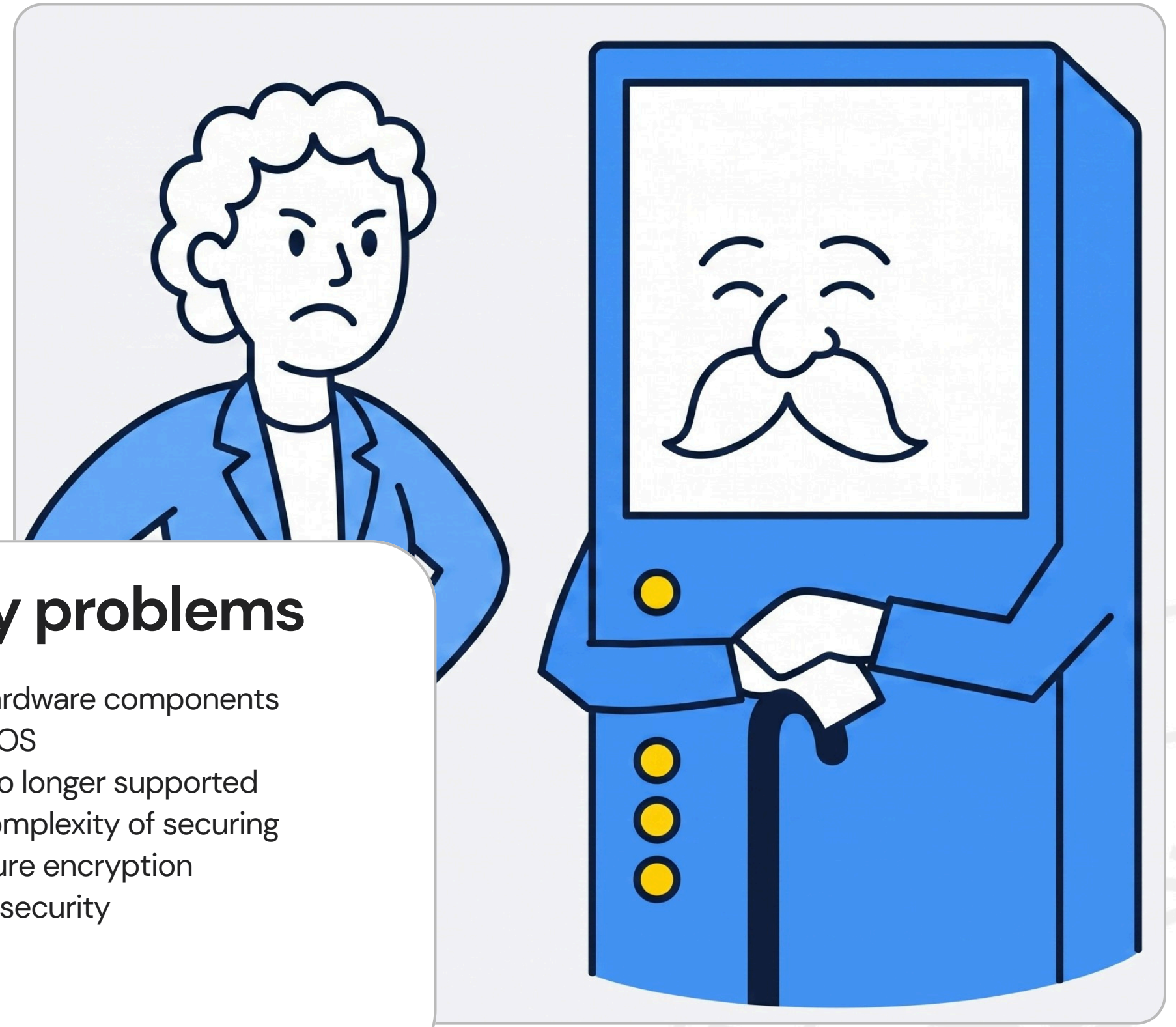
"Software or hardware that was built using methods that are outdated or obsolete"

Examples:

- Industrial robots
- PLC and SCADA systems
- Old mainframes
- OS used for specialised medical equipment

Security problems

- Obsolete hardware components
- Proprietary OS
- Softwares no longer supported
- Cost and complexity of securing
- Lack of secure encryption
- Safety over security



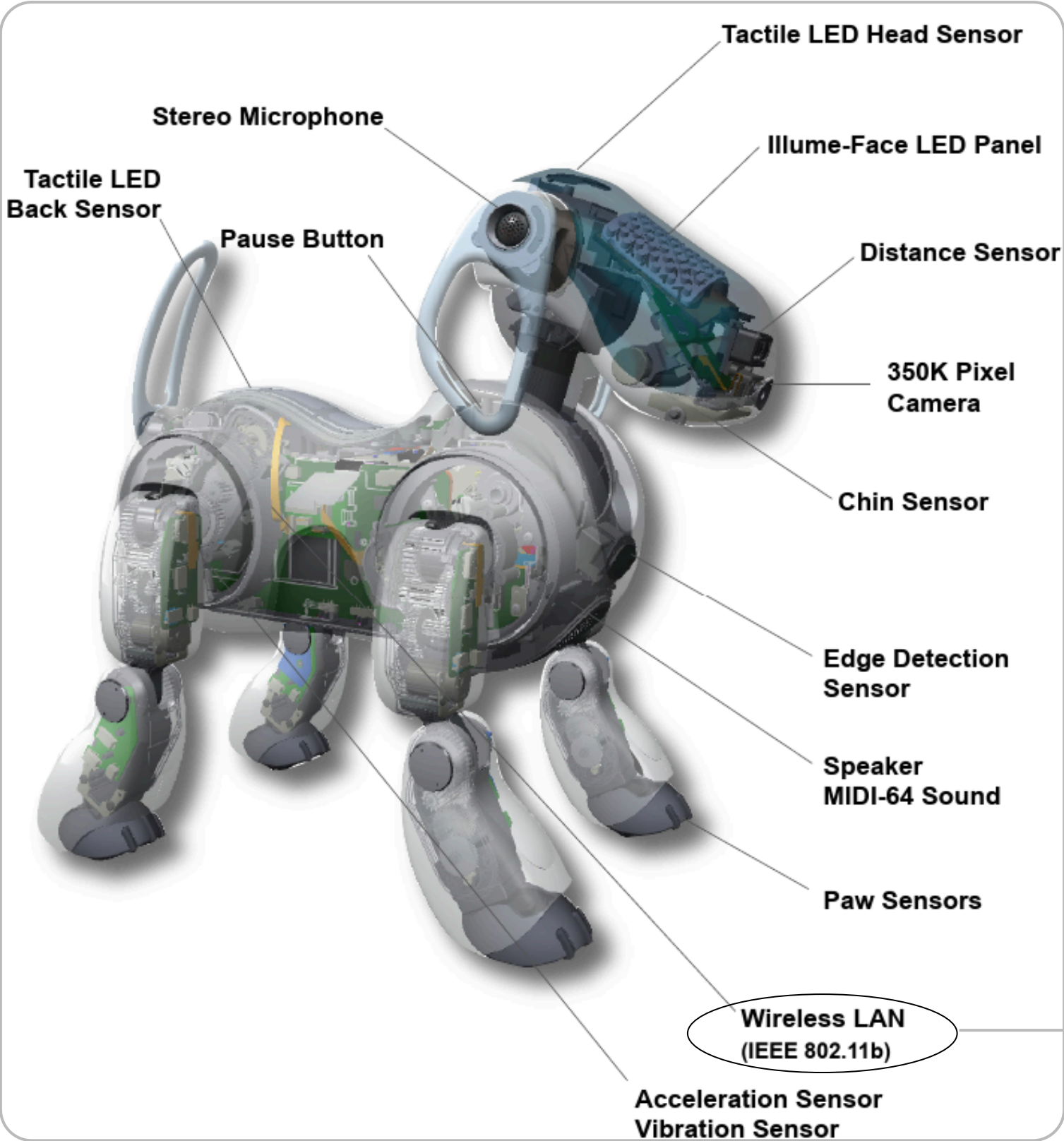
Sony's AIBO ERS-7



- MIPS processor
- RISC architecture
- Clocked at 576 MHz
- Secondary 256 KB cache on-chip



- 2200 mAh, runtime:
- Around 1.5 hours



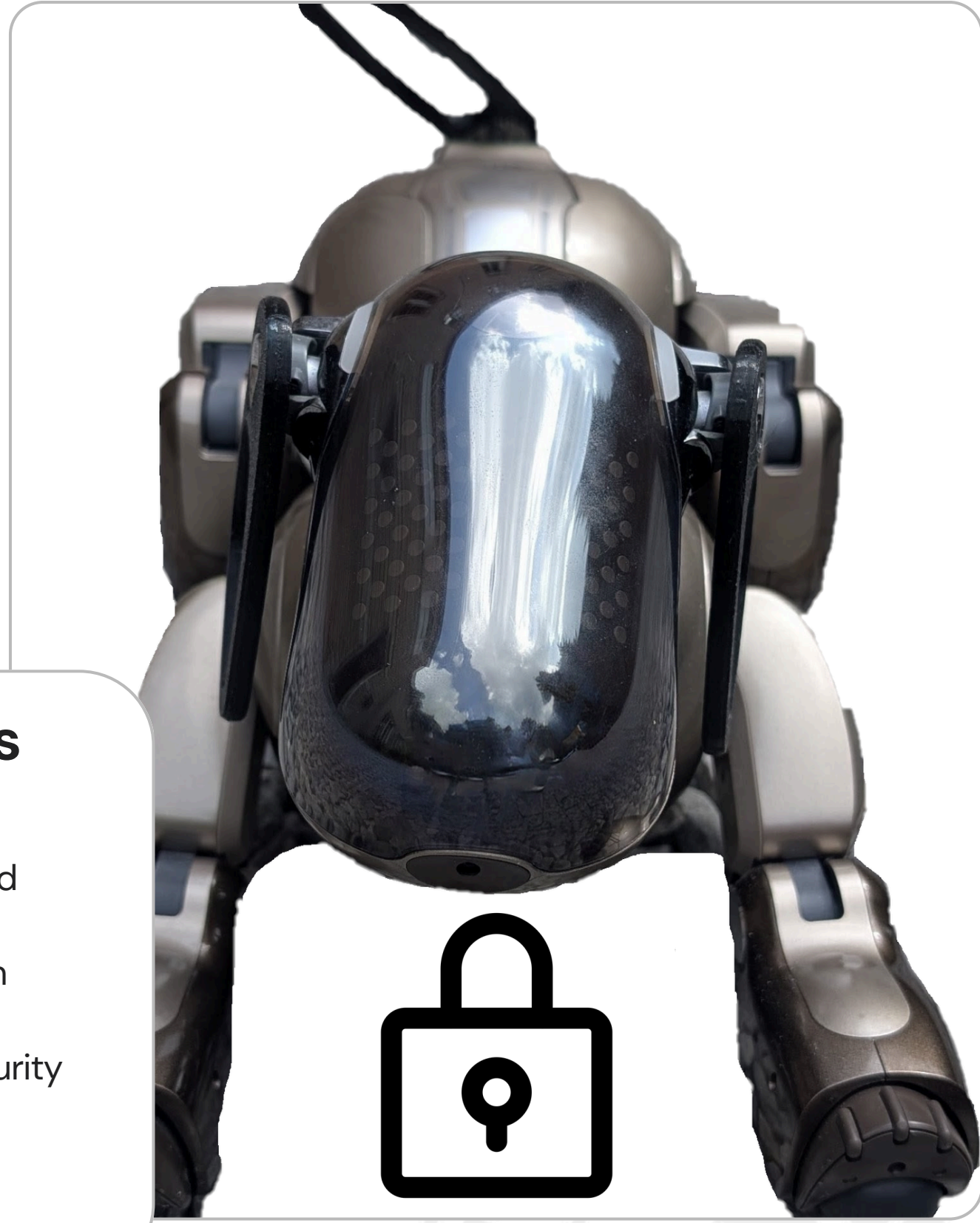
- 128 MB of storage:
- Data
 - Custom software

WEP (keys 64 or 128 bits)

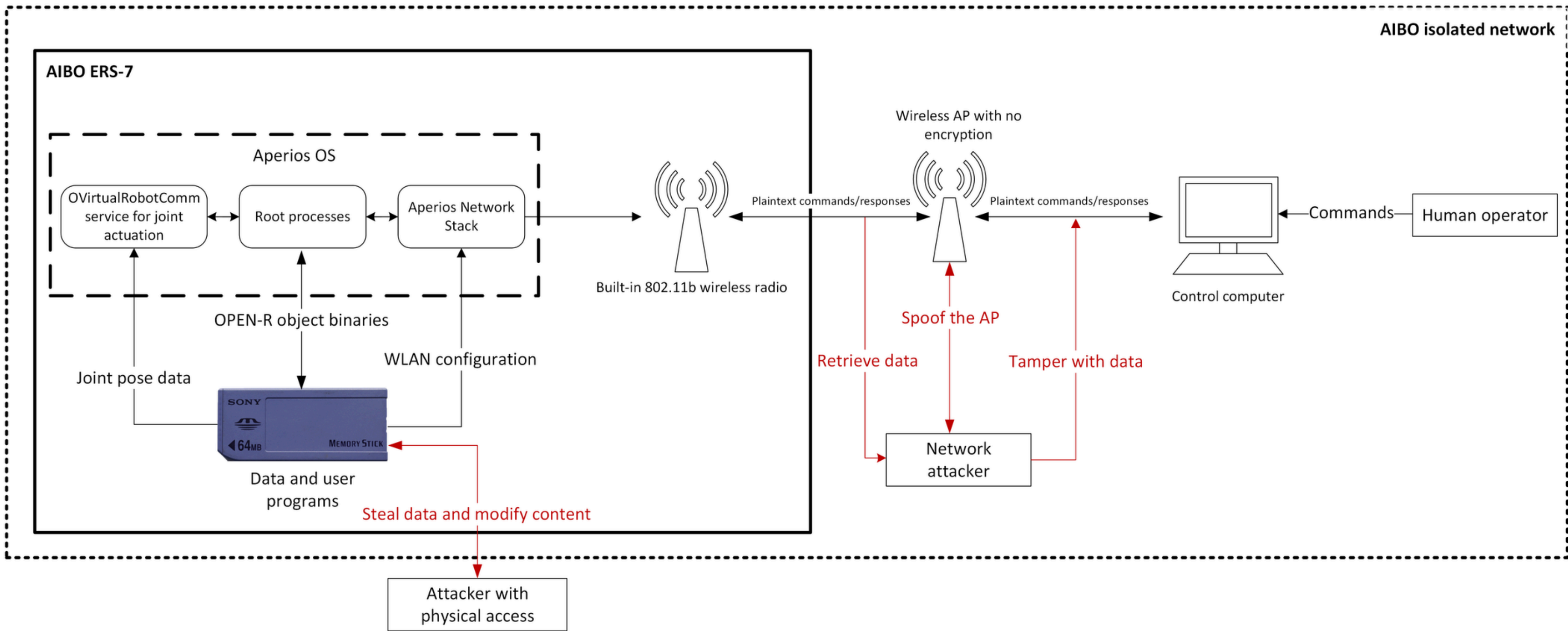
02. Problem statement

“Can the remote network control of a legacy robot be adapted using modern cryptographic techniques (confidentiality, integrity and mutual authentication) using only the platform’s original toolchain, and what are the security and performance limits of such a retrofit?”

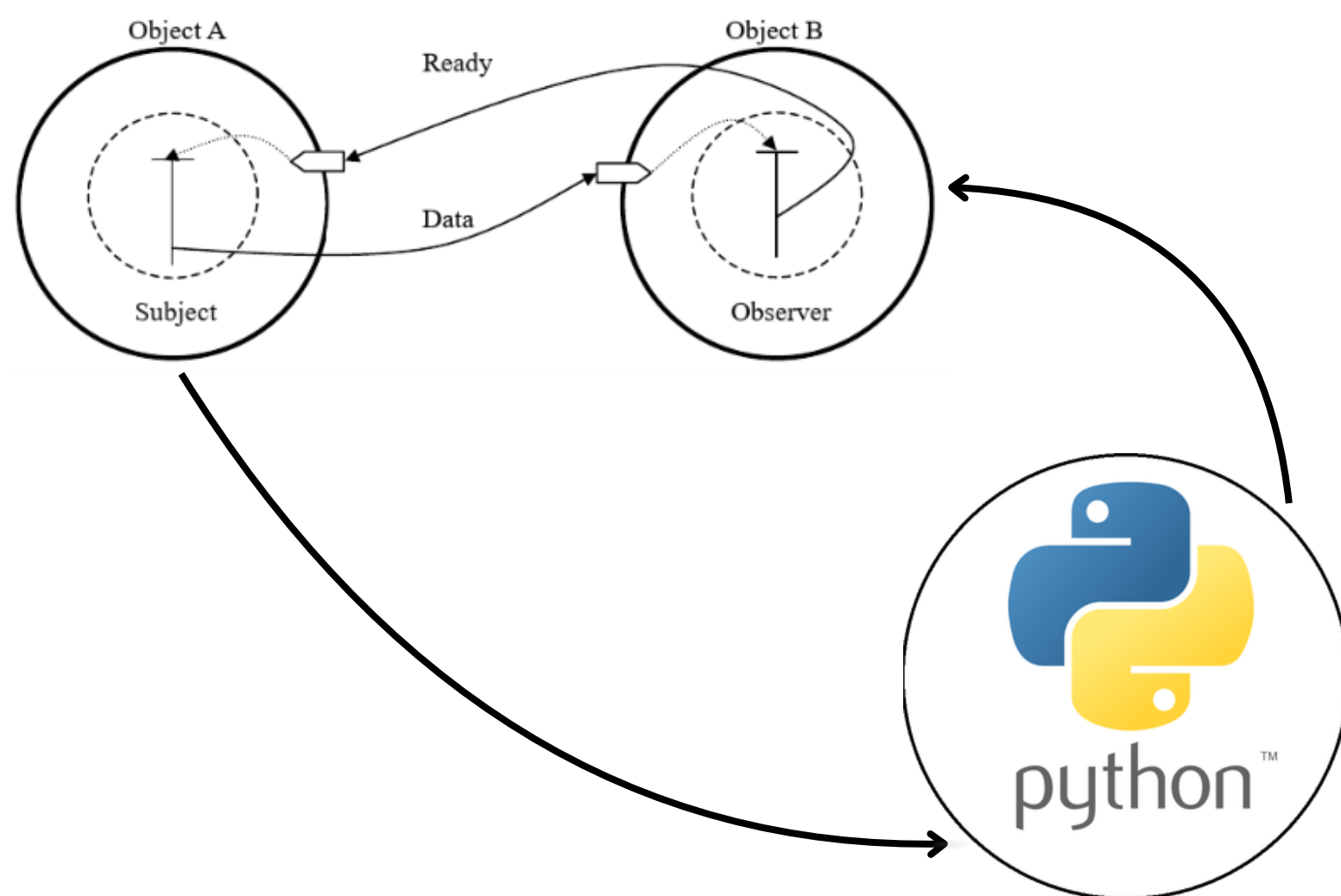
- This type of problem is not new!**
- In 2021, 73% of Healthcare still used legacy OS.
 - 6-axis IRB140 industrial robot from ABB preferred safety > security.
 - Extended lifetime = decline in security over time
 - Today’s cutting-edge becomes tomorrow’s liability.



03. Threat modeling



04. Secure solutions



Robot's side

OPEN-R object:

- C++
- C for cryptographic implementation

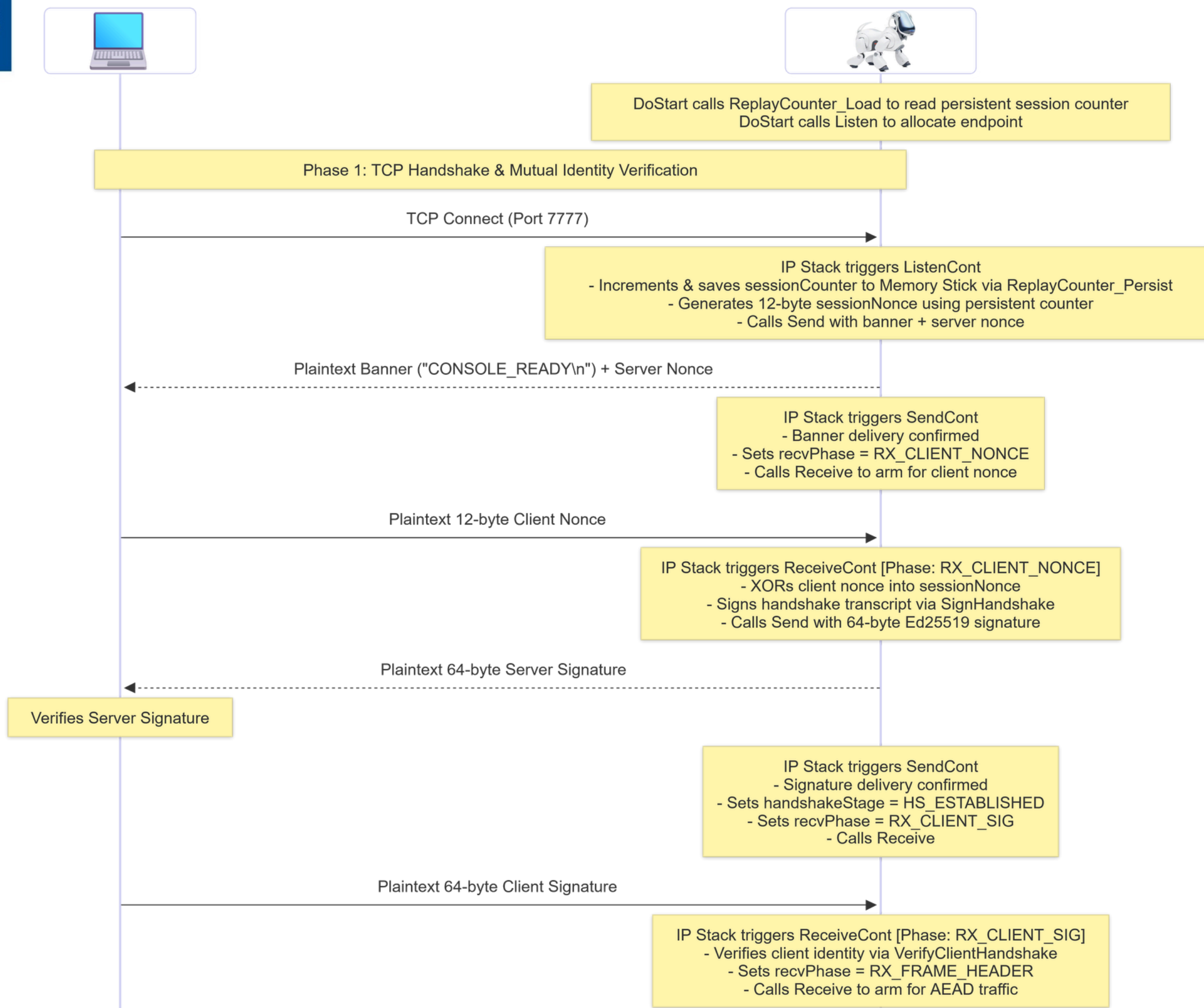
Handles:

- IP input/output
- Joint movements
- Cryptographic tasks (encryption, signature, MAC) and padding
- Session counter on Memory Stick

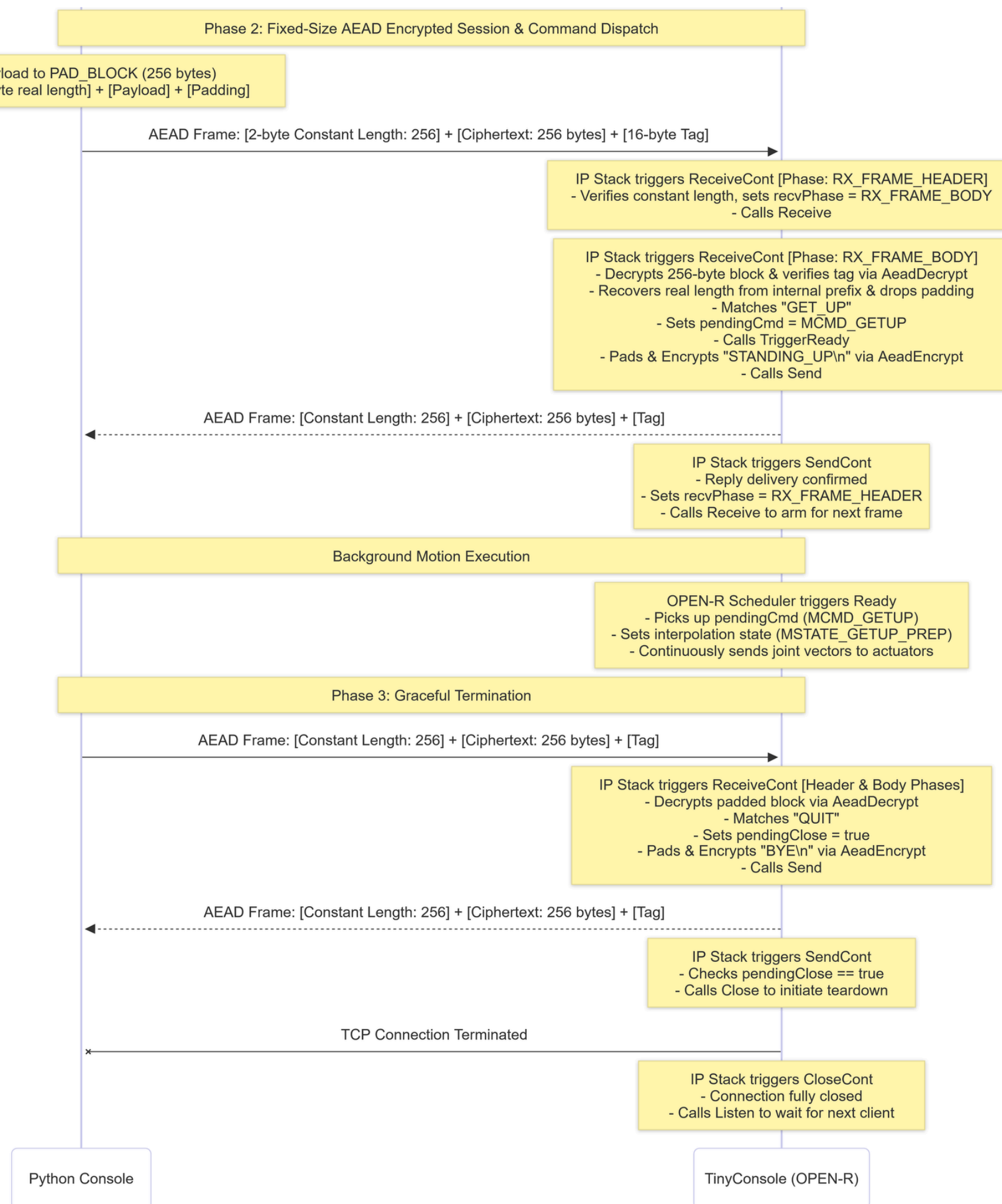
Controller's side

Python console program:

- Gives access to the user to a simplified control console
- Receives commands and output robot's answers
- Verification of local copy's session counter
- Mirror the robot's cryptographic operations
- In the event of a security error: fail and close.
- Tracks initialisation times and RTT



1. sessionCounter incremented & persistent
2. serverNonce = sessionCounter || client IP || 0000
3. sessionNonce[0:8] ⊕ os.urandom(12)[0:8]
4. Transcript = ("AIBO-TinyConsole-Handshake" || serverNonce || clientNonce)
5. Signature: robot → client
6. Signature: client → robot
7. Begin AEAD



8. Pad payload to 256 bytes

9. AEAD frame:

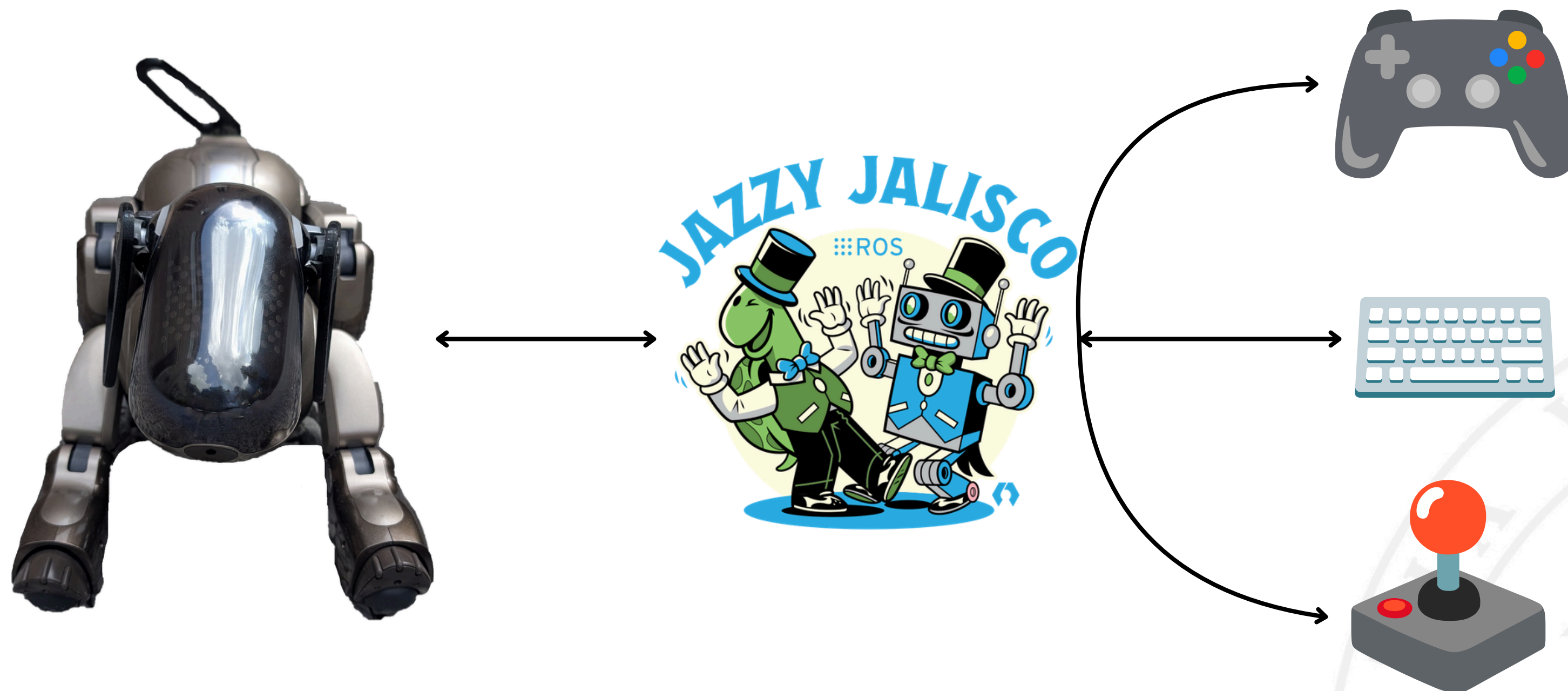
- header = size of padded (256)
- ciphertext = real length, payload, padding
- tag = Poly1305's MAC

9. Verify tag then decipher

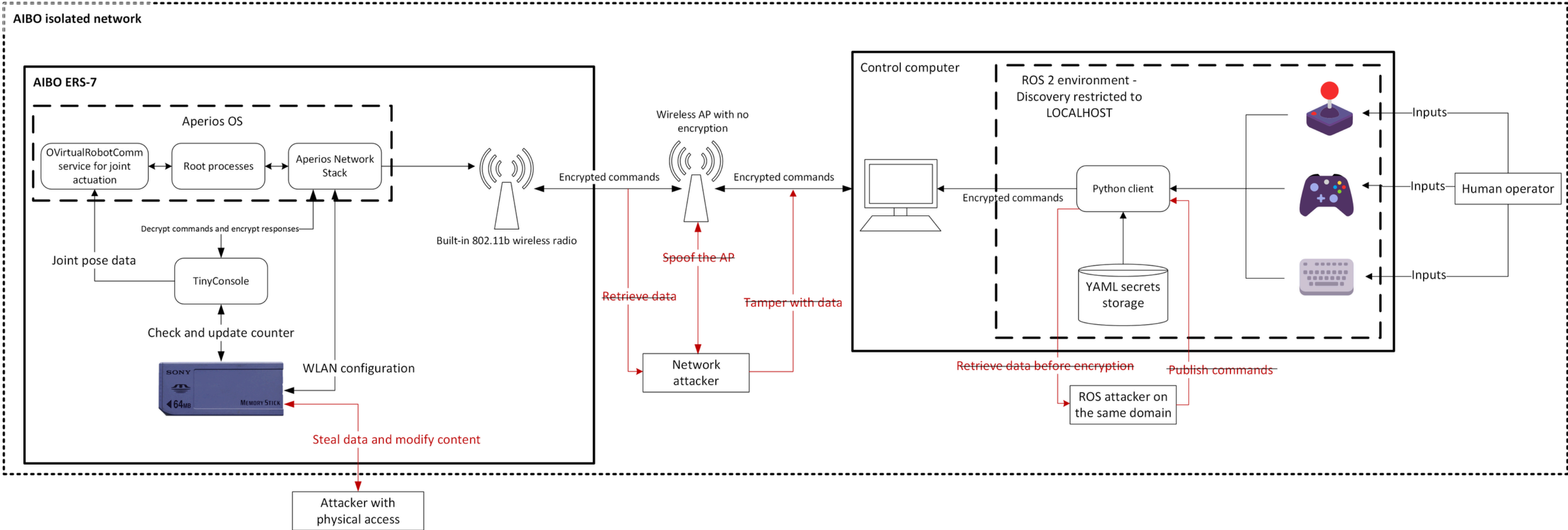
10. Execute commands

11. QUIT → BYE → CLOSED

05. ROS 2 Integration



State of threats



06. Testing

Replay attack

- Tries to replay previous valid conversation/command
- Robot verifies against its own frame per-direction counter/sessionCounter
- Robot rejects conversation/command
- Session closes
- No detailed error message sent to client

1

Test 1: Replay Attack (Duplicate First Command Frame)

Client

Valid command "HELP"



1. Valid command
2. Duplicate command

Robot



Spoofer AP captures, duplicates and sends the frame after the first valid command

Connection stopped (duplicate detected)

```
python3 './Python scripts\chacha20_console_client.py' -i 192.168.12.1 --port 7777
Connected to robot at 192.168.12.1:7777
Banner: CONSOLE_READY
Robot identity verified against ROBOT_PUBKEY_HEX.
Client identity signature sent (robot verifies against pinned key).
AIBO> HELP
ROBOT: Available commands: PING, GET_UP, REST, FORWARD, BACK, STOP, HELP, INFO, QUIT
AIBO> INFO
Robot closed the connection.
```

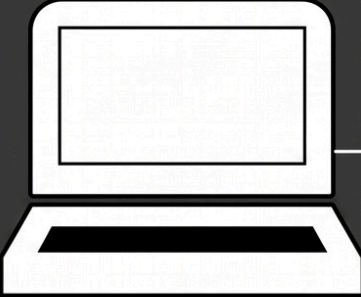

Integrity attack

- Modify tag (similar to modifying a message)
- Simulate attacker sending previous correct message with intercepted message's MAC
- Tag verification fails
- Connection stops
- No error message sent to client

2

Test 2: Integrity Attack (Bit-Flip in First Frame)

Client



Robot



Valid command "HELP"



Command with modified tag

Connection stopped (Poly1305 MAC Mismatch)

First Frame Data

16 byte tag

...

1

1

0

0

Attacker modifies AEAD tag

```
python3 './Python scripts\chacha20_console_client.py' -i 192.168.12.1 --port 7777
Connected to robot at 192.168.12.1:7777
Banner: CONSOLE_READY
Robot identity verified against ROBOT_PUBKEY_HEX.
Client identity signature sent (robot verifies against pinned key).
AIBO> HELP
Robot closed the connection.
```


3

Test 3: Handshake Tampering (Spoofed Key)

Client



Robot



Send plaintext clientNonce

Send plaintext clientNonce

Robot's signature of transcript

Spoofed signature

Modify 1 byte of signature to "generate" a new signature

Key mismatch

```
python3 './Python scripts\chacha20_console_client.py' -i 192.168.12.1 --port 7777
Connected to robot at 192.168.12.1:7777
Handshake failed: Robot identity verification FAILED. The device that answered did not sign the handshake with the expected key. This could mean an impersonator/MITM on the network, or that ROBOT_PUBKEY_HEX is stale after the robot was re-flashed with a new identity. Refusing to proceed.
```

- Handshake tampering
- Phase 1 of client $\leftrightarrow$ robot connection
 - Attacker tries to spoof the robot's signature (uses own public key)
 - Signature verification fails
 - Connection stops
 - Detected by the client $\rightarrow$ no round trip to the robot needed.

07. Measurements

```

sudo python3 nonce_audit.py --dir captures/
Audited 17 session(s) from captures/

[ok] session_prefix (64-bit): no reuse across 17 session(s)
[ok] robot_nonce (96-bit): no reuse across 17 session(s)
[ok] client_nonce (96-bit): no reuse across 17 session(s)

[ok] robot persistent counter strictly increasing (11 .. 34)

--- analytical bound ---
prefix space          : 2^64
sessions observed (k) : 17
P(any prefix collision) = k^2 / 2^65 = 7.833e-18
Reuse becomes ~50 percent likely only near k = 2^32 (4.3 billion) sessions.

RESULT: no nonce reuse observed; construction is sound.
Cannot be considered a proof that the scheme is secure

```

Verification of the nonce construction

Ciphertext analysis via ent

```
sudo python3 cipher_sample.py --dir captures/
[*] wrote 25088 bytes (98 ciphertext blocks) to cipher_sample.bin
[i] Note: this is a small sample. `ent` is fine here, but dieharder and NIST STS need many megabytes to
    be meaningful.

[*] running: ent cipher_sample.bin

Entropy = 7.992854 bits per byte.
// Maximum theoretical value = 8 bits/byte

Optimum compression would reduce the size
of this 25088 byte file by 0 percent.
// Indicates that there are no discernible patterns or redundant structures for an algorithm to
compress

Chi square distribution for 25088 samples is 247.82, and randomly
would exceed this value 61.47 percent of the times.
// Percentage between 10% and 90% = "pass"

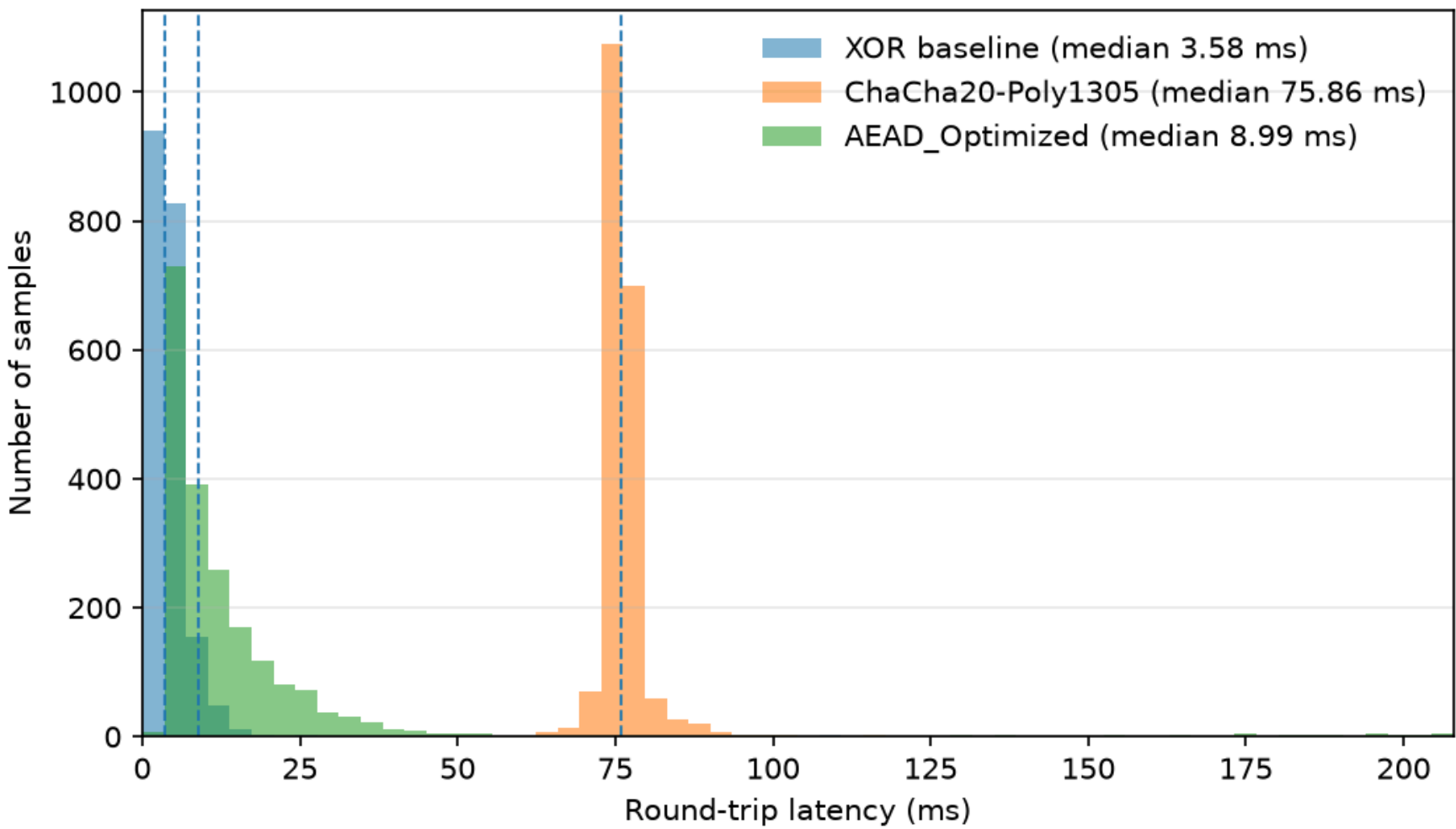
Arithmetic mean value of data bytes is 127.8723 (127.5 = random).
// Close to the value of a perfectly random dataset

Monte Carlo value for Pi is 3.147572351 (error 0.19 percent).
// Small error margin = spatial distribution of bytes is highly uniform

Serial correlation coefficient is -0.001412 (totally uncorrelated = 0.0).
// The closer to 0, the less there is a dependence between each byte.
```

Latency comparison

Metric	ChaCha20-Poly1305	AEAD, network only	XOR baseline
Samples	2000	2000	2000
Min (ms)	64.983	3.397	2.477
Mean (ms)	77.456	16.216	5.576
Median (ms)	75.860	8.987	3.583
p95 (ms)	80.894	35.193	9.735
p99 (ms)	95.085	197.208	16.770
Max (ms)	595.241	254.131	236.093
Std dev (ms)	19.271	29.044	14.106



08. Learning outcome

Can confidentiality and integrity be provided at application level within the ERS-7's limitations?

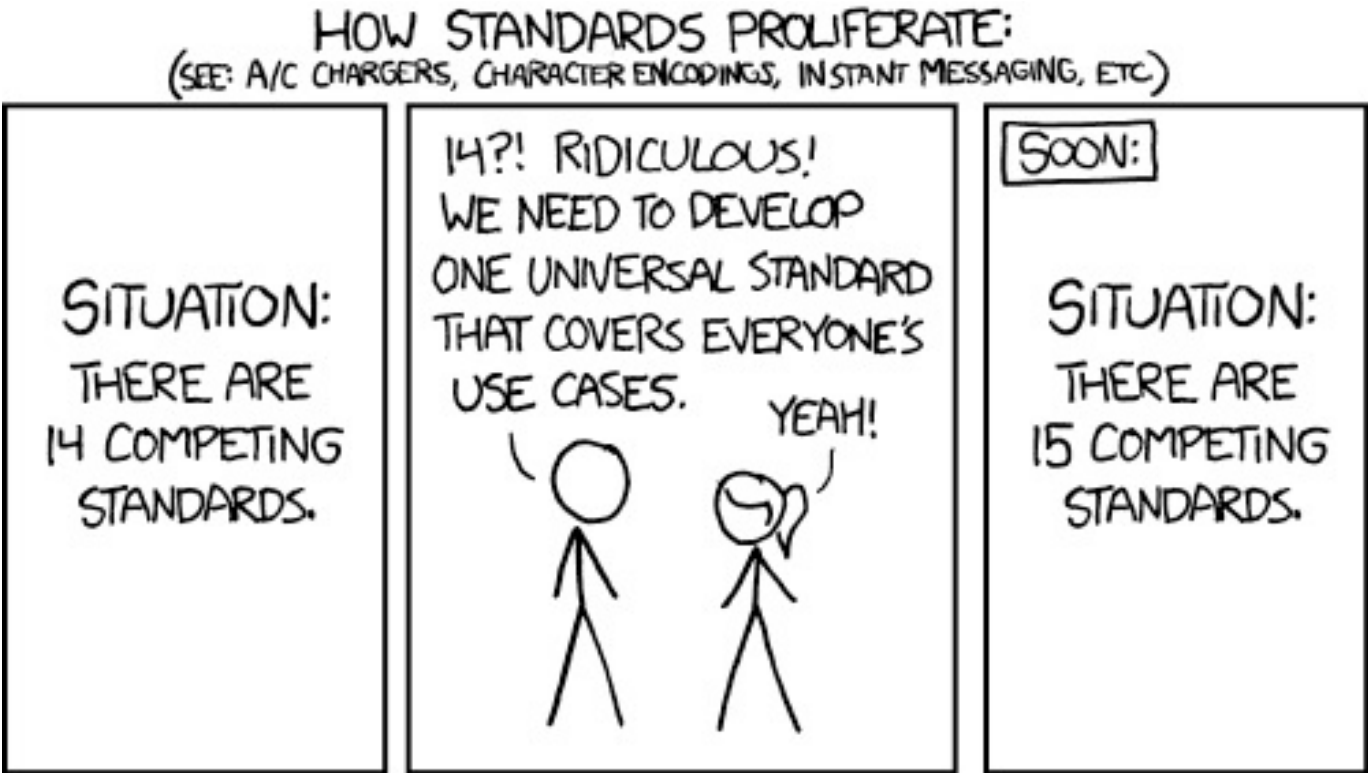
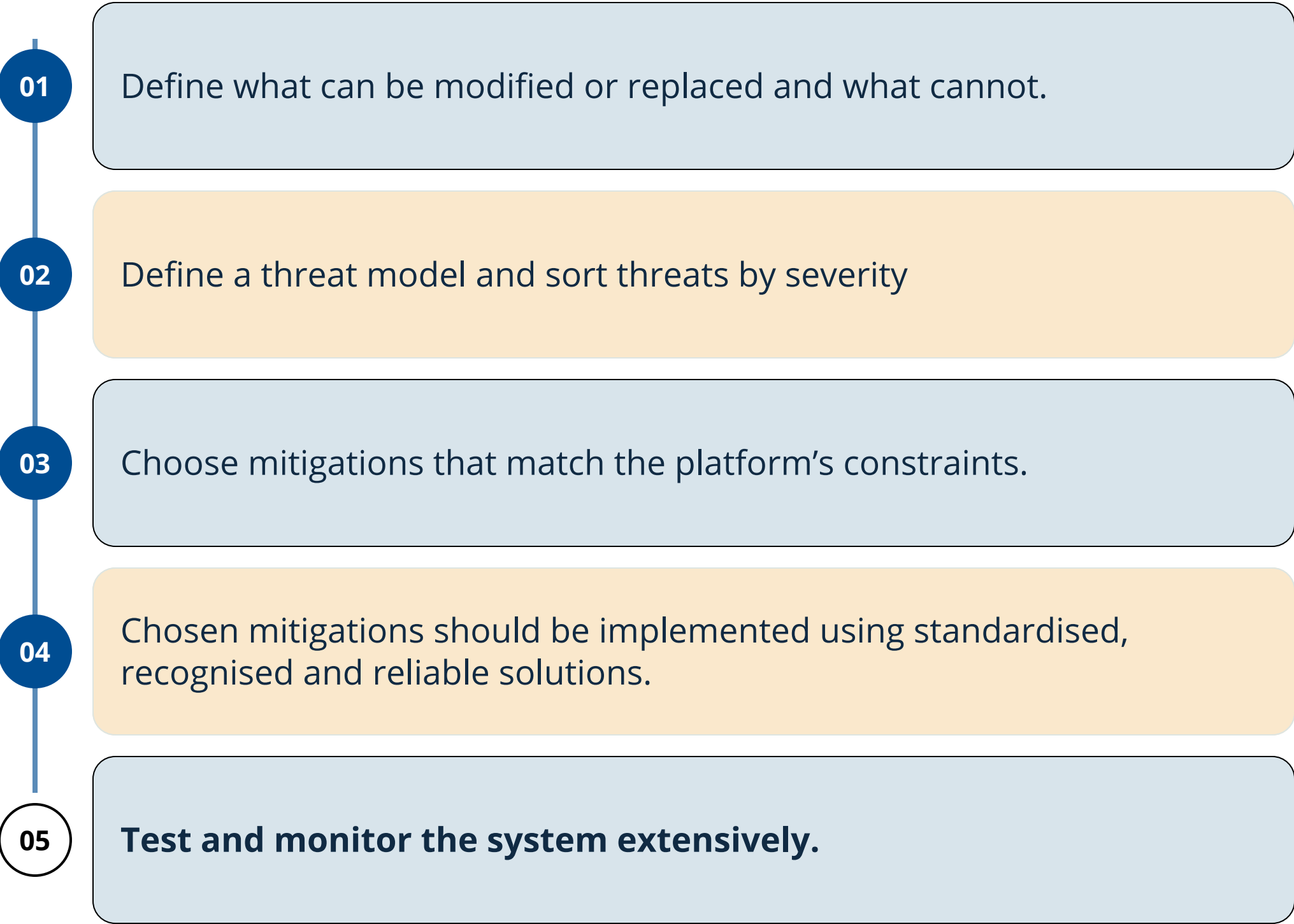
What is the cost of the retrofit in latency and footprint, and does it remain usable for real-time teleoperation?

Can the endpoints be mutually authenticated and what residual risks remain?

What is the impact of an attacker gaining physical access in terms of security?

Can the resulting secure remote control be interfaced with modern controllers using ROS 2's middleware?

Methodology to secure a legacy system



09. Future work

AIBO ERS-7

- Design an entropy source robust enough for a RNG.
- Based on RNG, make forward secrecy possible.
- Post-quantum migration for Ed25519.
- Key management, storage and rotation.
- Use of SROS 2 (Secure ROS 2)

Other legacy systems

Apply the previously defined methodology to a wide range of different types of legacy system with the same characteristics of the AIBO (available SDK, powerful hardware, low-level access).

Based on gathered data, refine the methodology.




```
PS >python3 .\chacha20_console_client.py -i 192.168.1.124 -p 7777
Connected to robot at 192.168.1.124:7777
Banner: CONSOLE_READY
|
```

Python console demo

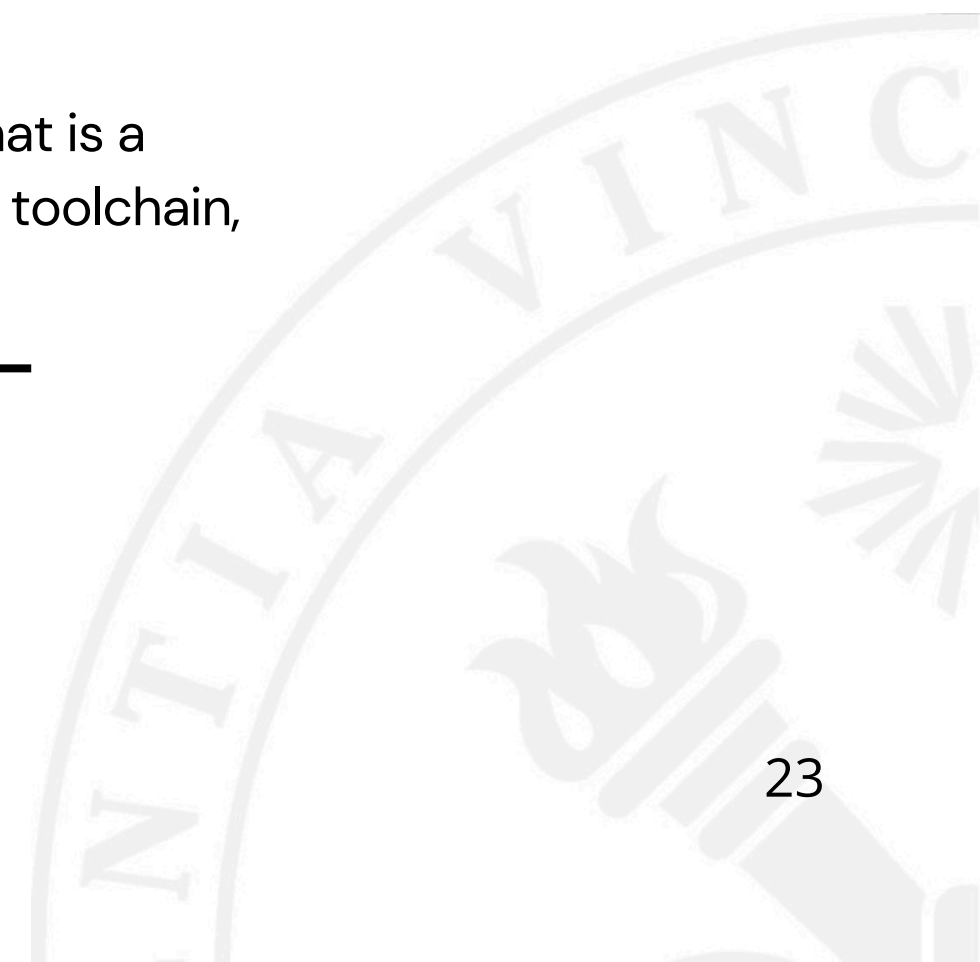
```
ubuntu@ubuntu2404: ~/Desktop/AIBO-Secure_Console/a... × ubuntu@ubuntu2404: ~/Desktop/AIBO-Secure_Console/a... × ubuntu@ubuntu2404: ~/Desktop/AIBO-Secure_Console/a... × ubuntu@ubuntu2404: ~/Desktop/AIBO-Secure_C
ubuntu@ubuntu2404:~/Desktop/AIBO-Secure_Console/aibo_bridge$ ros2 launch aibo_bridge aibo_bridge_launch.py robot_ip:=192.168.1.124
```

ROS 2 with keyboard demo



Thank You!

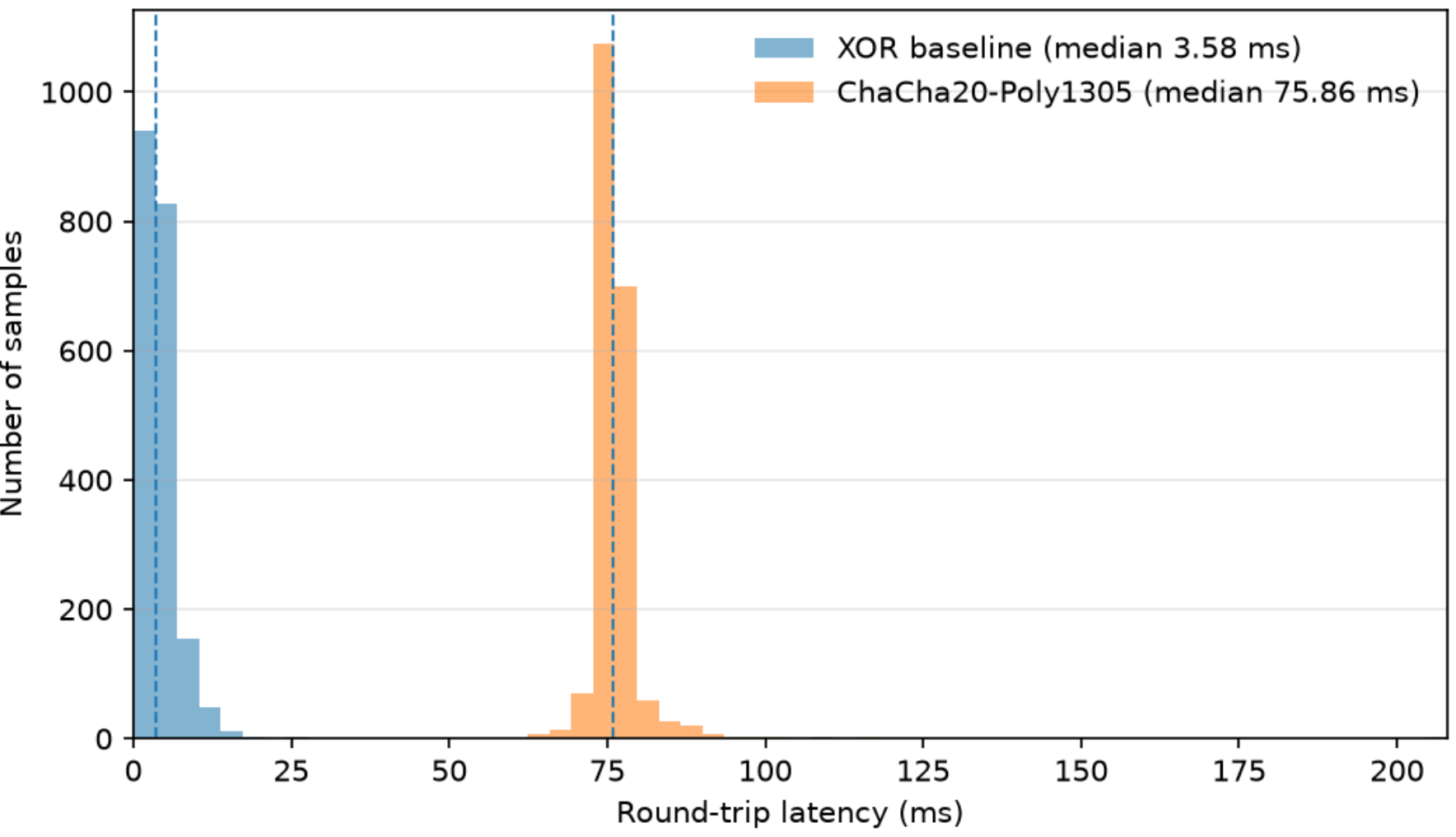
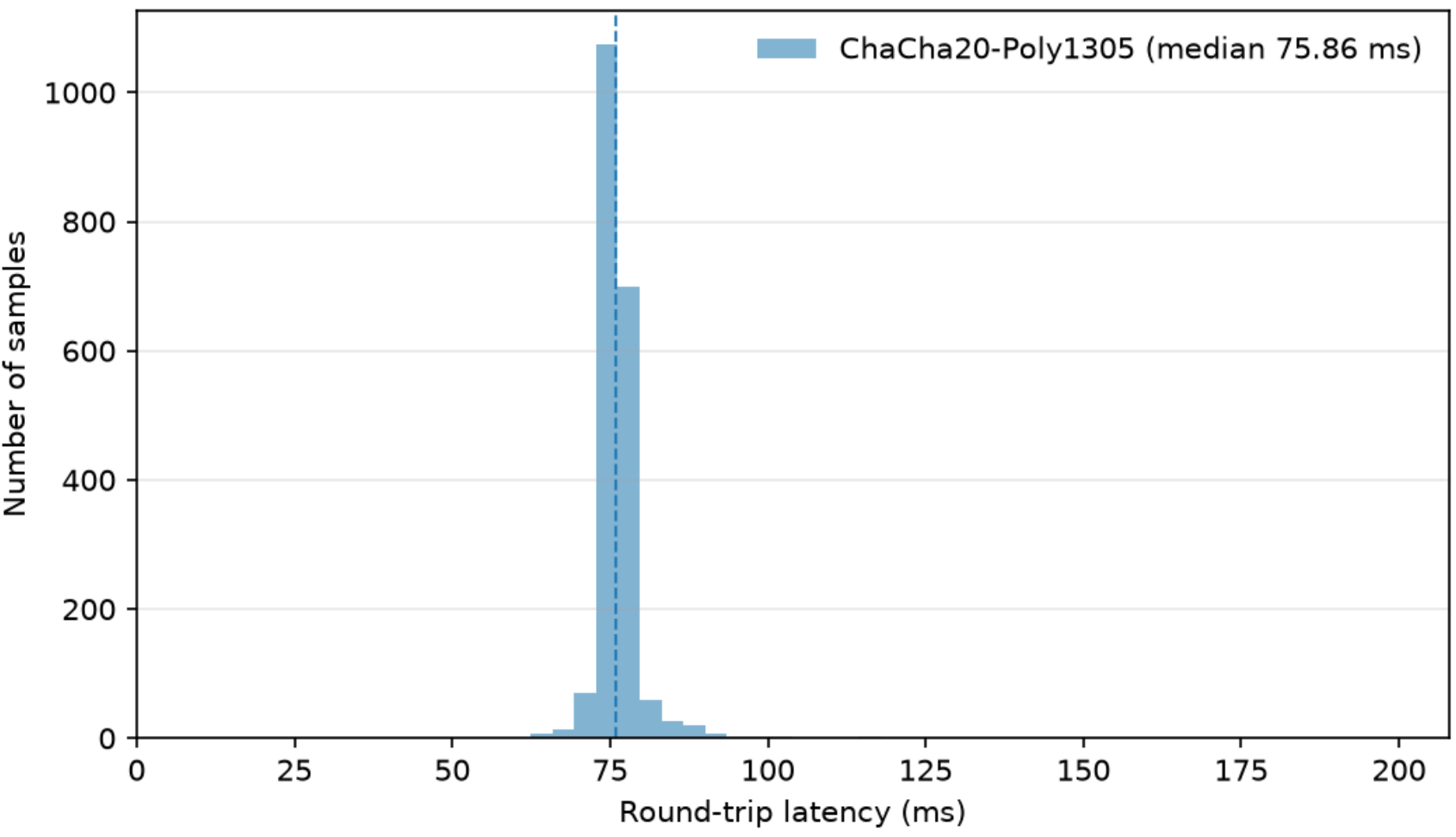
When a legacy system is decommissioned because it is no longer secure, that is a story, but when a legacy system's security is enhanced using only its legacy toolchain, that is a *good* story.



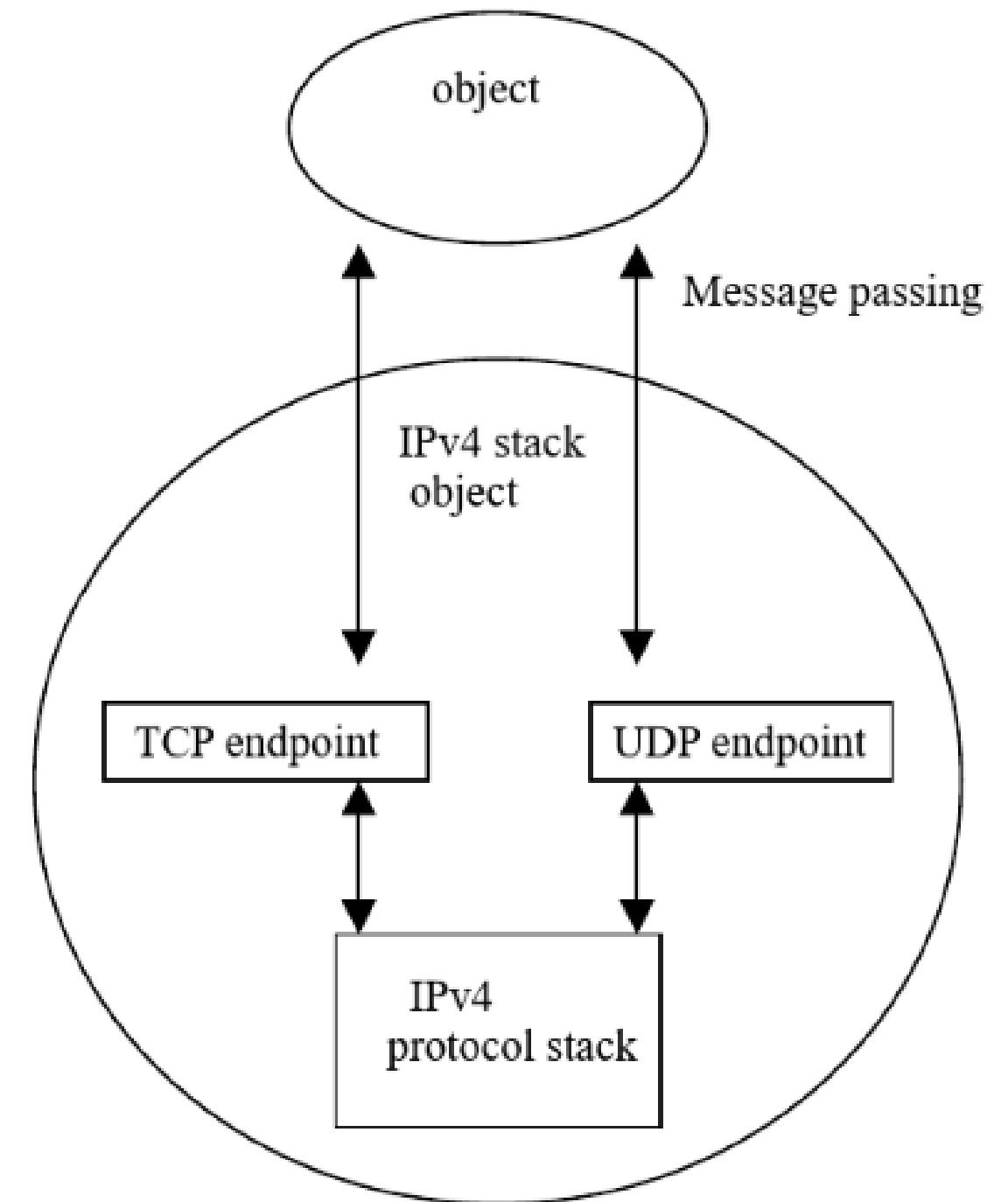
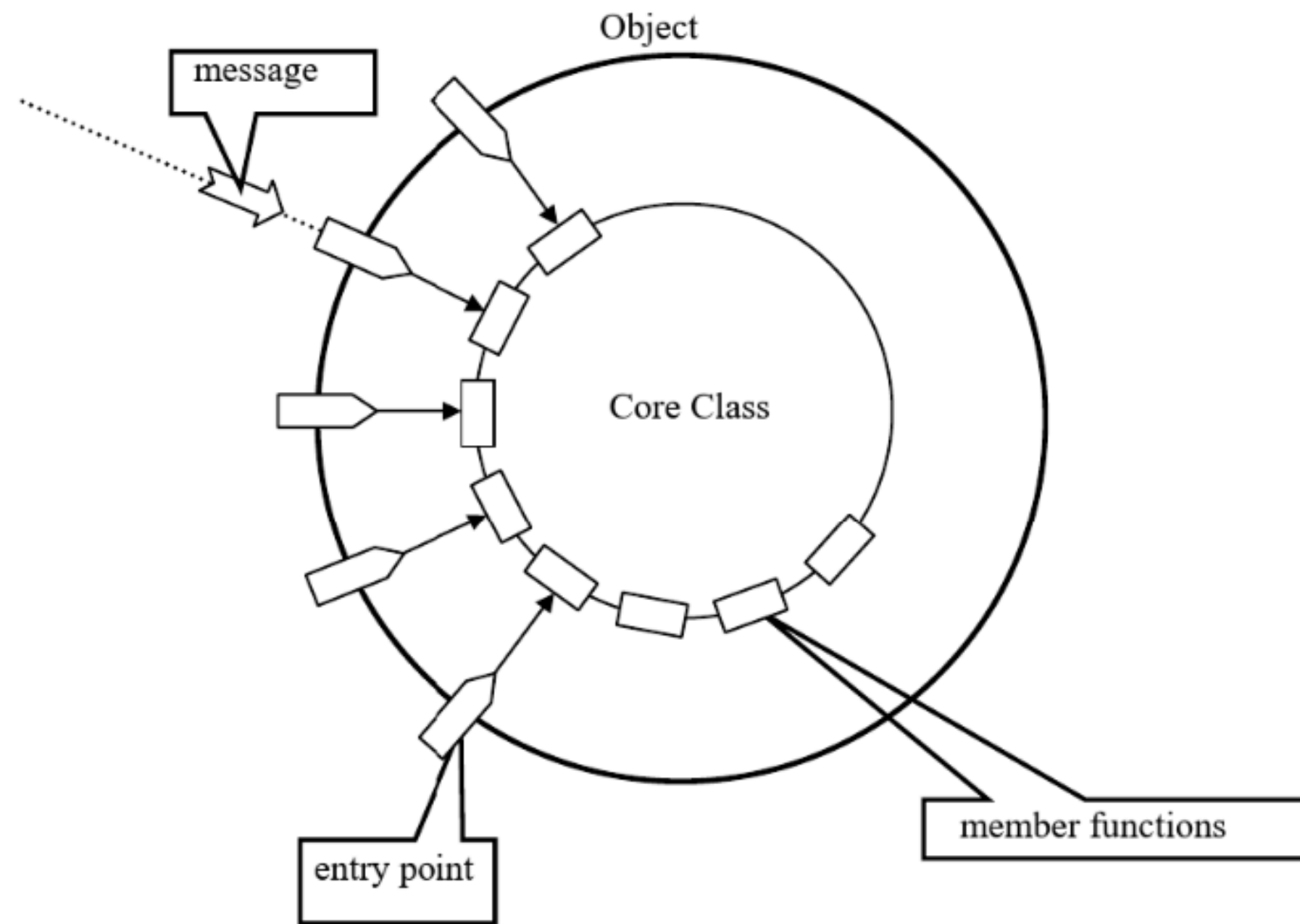
Appendix 1

		impact				
		trivial	minor	moderate	major	extreme
probability	rare	low	low	P1 & P2 & DS1	medium	DS2
	unlikely	low	low	DF3 & DF5	medium	medium
	moderate	low	medium	medium	medium	E1 & DF1
	likely	medium	medium	medium	DF2 & DF4	high
	very likely	medium	medium	high	high	high

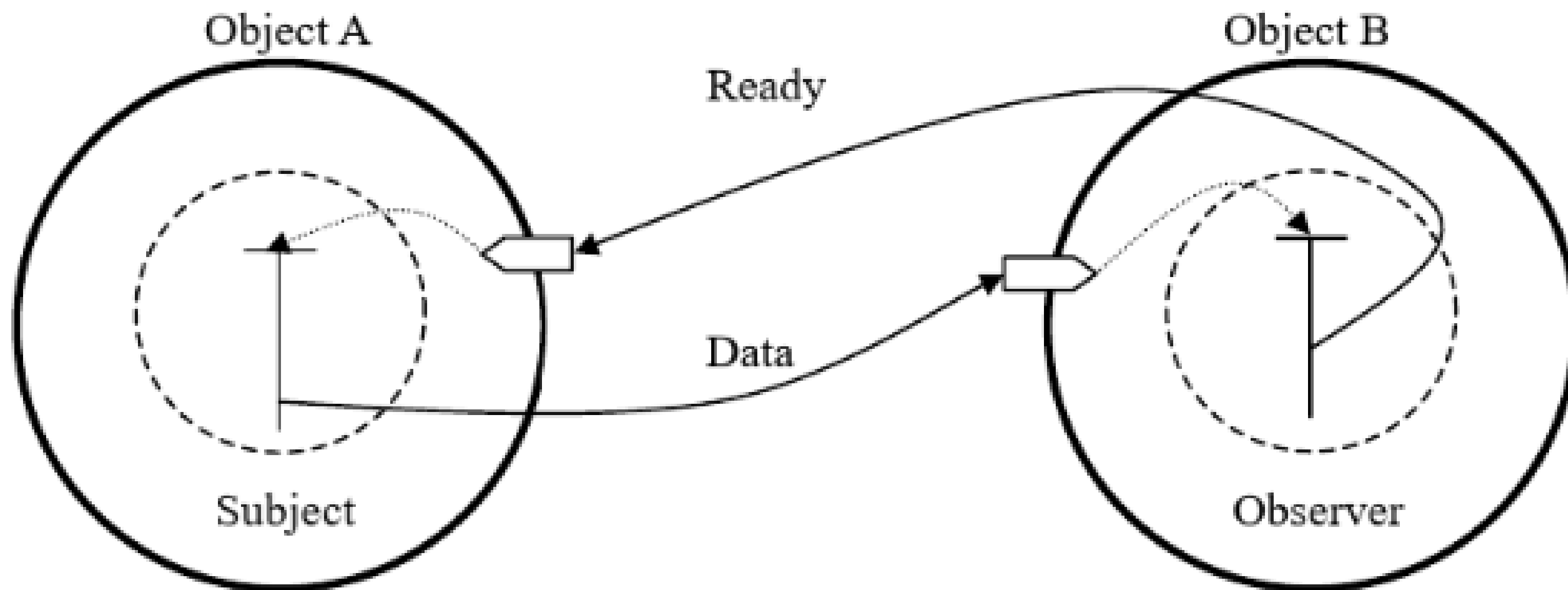
Appendix 2



Appendix 3



Appendix 4



```

OPENRSDK_ROOT?=/usr/local/OPEN_R_SDK
INSTALLDIR=./MS
CXX=$(OPENRSDK_ROOT)/bin/mipsel-linux-g++
CC=$(OPENRSDK_ROOT)/bin/mipsel-linux-gcc
STRIP=$(OPENRSDK_ROOT)/bin/mipsel-linux-strip
MKBIN=$(OPENRSDK_ROOT)/OPEN_R/bin/mkbin
STUBGEN=$(OPENRSDK_ROOT)/OPEN_R/bin/stubgen2
MKBINFLAGS=-p $(OPENRSDK_ROOT)
LIBS=-L$(OPENRSDK_ROOT)/OPEN_R/lib -lObjectComm -lOPENR -lInternet -lantMCOOP
CXXFLAGS= \
    -O2 \
    -g \
    -I. \
    -I$(OPENRSDK_ROOT)/OPEN_R/include/R4000 \
    -I$(OPENRSDK_ROOT)/OPEN_R/include/MCOOP \
    -I$(OPENRSDK_ROOT)/OPEN_R/include

CXXFLAGS+= -DOPENR_DEBUG
CFLAGS= -O2 -g -I. -DPOLY1305_32BIT

.PHONY: all install clean

all: tinyConsole.bin

%.o: %.cc
    $(CXX) $(CXXFLAGS) -o $@ -c $^

%.o: %.c
    $(CC) $(CFLAGS) -o $@ -c $^

TinyConsoleStub.cc: stub.cfg
    $(STUBGEN) stub.cfg

CRYPTO_OBJS = rfc7539.o chacha20poly1305.o poly1305-donna.o chacha_merged.o tweetnacl.o

tinyConsole.bin: TinyConsoleStub.o TinyConsole.o replay_counter.o $(CRYPTO_OBJS) tinyConsole.ocf
    $(MKBIN) $(MKBINFLAGS) -o $@ $^ $(LIBS)
    $(STRIP) $@

install: tinyConsole.bin
    gzip -c tinyConsole.bin > $(INSTALLDIR)/OPEN-R/MW/OBJS/TINYCONS.BIN

clean:
    rm -f *.o *.bin *.elf *.snap.cc
    rm -f TinyConsoleStub.h TinyConsoleStub.cc def.h entry.h
    rm -f $(INSTALLDIR)/OPEN-R/MW/OBJS/TINYCONS.BIN

```

Appendix 5

Makefile

- Define files locations
- Define compiler flags
- Define which compiled .cc files must be created
- Define final compressed binary (8.3 filename)
- Make operations

```

OStatus
TinyConsole::DoInit(const OSystemEvent& event)
{
    OSYSDEBUG(("TinyConsole::DoInit()\n"));

    // Macros defined in <OPENR/core_macro.h>
    NEW_ALL_SUBJECT_AND_OBSERVER; // Registers the necessary number of observers and subjects
    REGISTER_ALL_ENTRY; // Registers the connection to services offered by other objects
    SET_ALL_READY_AND_NOTIFY_ENTRY; // Registers the entry points for receiving messages

    OpenPrimitives();
    NewCommandVectorData();

    OPENR::SetMotorPower(opowerON);
    // Necessary to power on motors and must be called in this function
    return oSUCCESS;
}

OStatus
TinyConsole::DoStart(const OSystemEvent& event)
{
    // ..... //
    // Initialisation of ANT stack //
    // ..... //
    if (subject[sbjMove]->IsReady() == true) {
        double cur[NUM_JOINTS];
        ReadCurrentPose(cur);
        SetupInterpolation(cur, SLEEPING_POSE_3, STARTUP_COUNTER);
        // Used to move the joints to the sleeping pose with a counter which indicates the time that
        this action takes, value must be adjusted to avoid any sudden movements; robot only be switched on when
        it is already in a seated position

        SetJointGain(); // Position of this function is important as if it isn't called at the "good"
        time,
        // joints won't move at all when receiving commands
        motionState = MSTATE_STARTUP;
        AdvanceInterpolation(STARTUP_COUNTER); // Generate the 1st 128ms block of joint movements
        // Sends it to the motor subject sbjMove; once this has finished the scheduler will ask for the
        next frame
    } else {
        motionState = MSTATE_IDLE; // In the case the motor observer is not ready/has crashed ->
        avoid sending data which could cause an exception
    }

    // Macros defined in <OPENR/core_macro.h>
    ENABLE_ALL_SUBJECT; // Enables the subjects of the core class
    ASSERT_READY_TO_ALL_OBSERVER; // Sends ASSERT_READY to all connecting subjects (as previously
    defined)

    return Listen();
}

```

Appendix 6

DoInit:

- Initialises object
- Creates observers
- Sends power to motors

DoStart:

- Sets joints to their initial position
- Start motion state machine
- Enable subjects.

Appendix 7

```
static const char* const JOINT_LOCATOR[TinyConsole::NUM_JOINTS] = {
    "PRM:/r4/c1-Joint2:41",      // Right Front J1
    "PRM:/r4/c1/c2-Joint2:42",   // Right Front J2
    "PRM:/r4/c1/c2/c3-Joint2:43", // Right Front J3

    "PRM:/r2/c1-Joint2:21",      // Left Front J1
    "PRM:/r2/c1/c2-Joint2:22",   // Left Front J2
    "PRM:/r2/c1/c2/c3-Joint2:23", // Left Front J3

    "PRM:/r3/c1-Joint2:31",      // Left Rear J1
    "PRM:/r3/c1/c2-Joint2:32",   // Left Rear J2
    "PRM:/r3/c1/c2/c3-Joint2:33", // Left Rear J3

    "PRM:/r5/c1-Joint2:51",      // Right Rear J1
    "PRM:/r5/c1/c2-Joint2:52",   // Right Rear J2
    "PRM:/r5/c1/c2/c3-Joint2:53", // Right Rear J3
};

static const double SLEEPING_POSE_0[12] = {
    10, 1, 30, // Front
    10, 1, 30, // Front
    -35, 5, 60, // Rear
    -35, 5, 60, // Rear
};
```

Assignment of joint values

